ns3-continuum: Bridging Network and Compute Simulation for Distributed Computing Systems

John Mullan
School of Computer Science
University College Cork
Cork, Ireland
122331816@umail.ucc.ie

Ahmed H. Zahran
School of Computer Science
University College Cork
Cork, Ireland
ahmed.zahran@ucc.ie

***Abstract*—Evolving applications feature demanding AI computing workloads and tight delay budgets for seamless user experience. These workloads are typically offloaded to distributed computing continuum tiers to meet their strict latency requirements. Scalable performance evaluation of these systems is beyond the reach of analysis and experimentation, leaving simulations as the key approach to testing the performance of these applications. However, existing simulation tools focus on either the network dynamics or computing operation or lack features required by complex workloads and advanced computing platforms. ns3-Continuum extends Network Simulator 3 (ns-3) to enable scalable performance evaluation of distributed computing systems supporting adaptive client behavior, composite workloads, and state-of-the-art accelerators. Our implementation is designed to facilitate tuning and extending end-device behavior, accelerator operation, and control plane decision engines. Our performance evaluation illustrates benchmarking the performance of distributed computing in networking contexts.**

***Index Terms*—Compute Continuum, MEC, task workflow, Offloading, adaptive, accelerator, GPU, scheduling, simulations.**

I. Introduction

The convergence of distributed computing and next-generation wireless networks promises to revolutionize latency-critical applications, such as extended reality, Industry 5.0 systems [1], and context-aware personal assistants. These applications exhibit complex processing pipelines that propels offloading computational workloads to edge tiers of the computing continuum. However, this promise is fundamentally predicated on fluctuations in the wireless medium and the contention to shared computing and communication resources. Hence, service operators consider advanced offloading strategies to heterogeneous continuum tiers while ensuring the satisfaction of service quality of service requirements [2], [3]. Hence, a robust performance evaluation of these systems evolves as a critical requirement before their scalable deployments.

Analytical queueing and optimization models can provide foundational insights, but they rely on simplified assumptions that systematically underestimate tail behavior emanating from underlying protocol, communication, and computing dynamics. Conversely, physical testbeds, though invaluable, are intrinsically prohibitive to scale and critically irreproducible, since no two experimental runs encounter identical channel realizations. Hence, simulation emerges as an indispensable approach that offers controlled, repeatable, scalable experimentation for evaluating the performance of distributed computing systems (DCSs).

Su et al. [4] classified distributed computing simulators into three categories: compute-based, network-based, and greenfield simulators. Compute-based simulators focus on modeling computation but, as a result, lack precise modeling of network aspects such as packet transfer. Network-based simulators focus on modeling the network and the various protocols that operate on it. These simulators often have limited support for computation-level dynamics such as task offloading and resource management. Greenfield simulators are built from scratch to focus on the simulation of a specific paradigm. Hence, the lack of generic DC simulators motivates development of new tools. However, these tools do not support complex workload (e.g., multiple computing tasks per offloaded requests) and advanced hardware features, such as dynamic voltage frequency scaling (DVFS).

In this paper, we present *ns3-Continuum*[1]: a highly extensible and configurable ns3 module for simulating distributing computing systems. The module enables simulating systems with unique features including complex workloads that are modeled using directed acyclic graphs, dynamic vertical GPU scaling, and adaptive client behaviour. These features extends ns3 network simulations to enable the evaluation of the application performance in DCS with advanced client and server behaviour. Our performance evaluation illustrates such capabilities by simulating edge assisted augmented reality over WiFi and complex workloads offloading in multi-tier DCSs.

The rest of this paper is organized as follows. Section II discusses background and related work. Section III outlines the design and implementation of ns3-Continuum, Section IV presents the simulation and evaluation of compute continuum workloads within ns3-Continuum. Finally, Section V concludes and possible future work is outlined.

II. Background and Related Work

Evolving DC Applications, such as XR and autonomous vehicles, typically exhibit composite inference pipelines involving multiple, interdependent machine learning models that

[1]ns3-continuum: https://github.com/MullanJohn/ns3-distributed

must execute in tight temporal cohesion. To illustrate, a typical augmented reality (AR) pipeline [5], [6] may concurrently run a lightweight object detector for scene segmentation, a pose-estimation network for user head and hand tracking, a depth-mapping model for occlusion handling, and a generative upsampler for rendering photorealistic overlays—all while fusing inertial measurement unit (IMU) data at millisecond granularity. These models contend for scarce computing resources, and their intermediate outputs often serve as inputs to downstream pipeline blocks, creating cascading timing constraints. This intricate coupling between application-level model dependency graphs and system resource utilization requires powerful tools that capture these details to enable evaluating advanced offloading strategies in DCSs.

DCS Accelerators come in different flavors with different capabilities [7]. GPUs provide massive parallelism and high throughput for data-parallel workloads but have limited energy efficiency; FPGAs strike a flexible middle ground, enabling reconfigurable, application-specific pipelines with superior energy-per-operation than GPUs, albeit at the cost of challenging hardware description languages and lower peak frequency; ASICs achieve the highest performance and best energy efficiency for dedicated functions, but their inflexibility, long design cycles, and prohibitive fabrication costs make them viable only for selected applications. These accelerators can be generally simulated as a queuing system whose server defines the device processing capacity. However, some accelerators have unique features that require custom implementation. For example, GPUs support advanced features such as Dynamic Voltage and Frequency Scaling (DVFS) [8] that can be leveraged to optimize system energy consumption by adjusting the frequency and voltage. Supporting such features enables exploring state-of-the-art accelerator hardware.

DC Simulators vary in their capabilities and abstraction. SimGrid [9] represents a versatile, robust framework for evaluating scheduling and resource management algorithms for large-scale distributed systems in both cloud and HPC environments. Similarly, CloudSim [10] represents a widely used simulator for cloud computing virtualized infrastructures, but it is not designed for the edge continuum. iFogSim [11] extends CloudSim to simulate resource management in edge-fog environments, enabling studies of latency, network congestion, and energy consumption. EdgeCloudSim [12] addresses CloudSim's mobility and networking gaps through a modular design, yet facilitates module substitution at the expense of flexible inter-module interaction reconfiguration. The focus of these simulators remains predominantly computational, with limited support for relevant mobility and networking details.

To overcome this critical limitation, built on Network Simulator 3 (ns-3), EdgeSim++ [13] improves upon prior simulators by supporting customizable, dynamically-attribute tasks and device interactions. EasiEi [4]: ns-3-based (3.35); microkernel architecture with independent components (sender, receiver, orchestrator, generator); supports centralized/decentralized paradigms; components interact via Create Read Update and Delete (CRUD) on a task status table—a design that poorly models realistic information exchange.

TABLE I: Simulator and module capability comparison

A: Heterogeneous Network Support - B: Device Mobility Modelling - C: Dynamic Task and Resource Representation - D: Complex Pipelined Task Modelling - E: Accelerator-Aware Compute Modelling - F: Highly Configurable Client Abstraction - G: Energy/Power Modelling - H: Vertical Processing Scaling

Simulator / Module	A	B	C	D	E	F	G	H
ns-3	✓	✓	✗	✗	✗	✗	●	✗
OMNeT++	✓	✓	●	✗	✗	✗	●	✗
CloudSim	✗	✗	✓	●	✗	●	●	✓
SimGrid	●	●	✓	✓	✗	✓	✓	✓
iFogSim	●	●	✓	✓	✗	✗	●	✗
EdgeCloudSim	●	✓	●	✗	✗	●	✗	●
EasiEI	✓	✓	✓	✗	✗	●	●	✗
EdgeSim++	✓	✓	✓	✗	✗	●	●	✗
ns-3 Continuum	✓	✓	✓	✓	✓	✓	✓	✓

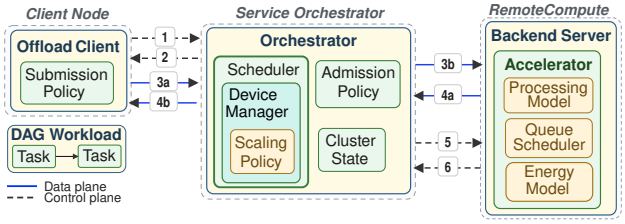


Fig. 1: ns3-continuum Architecture

Table I compares ns3-continuum with existing simulators considering eight different capabilities that are highly relevant to distributed computing.

III. Design and Implementation

A. ns3-Continuum Overview

ns3-Continuum simulates DCSs by implementing four components: the workload defined by Directed Acyclic Graph (DAG) of computing tasks, a configurable adaptive client, an orchestrator, and an abstraction for compute accelerators. Fig. 1 illustrates the architecture of the system and the typical interaction between its components. Clients send admission requests for their tasks to the orchestrator (1); the latter takes scheduling decisions assigning the task elements to worker nodes and sends admission outcome to the client (2); the client then forwards the task data to the orchestrator (3a), The orchestrator forwards the data (and intermediate output for composite workload) to the involved workers (3b). When the workload processing is complete, the orchestrator receives the processing results from the worker (4a) and forwards the final output to the client (4b). The orchestrator scheduler may also send further control plane instructions to the worker accelerator (e.g., energy management and scaling decisions as illustrated by (5) and (6) in Fig. 1). ns3-Continuum design prioritizes extensibility through reusable interfaces, allowing users to incorporate custom fields and behaviors rather than adhering to rigid, predefined formats. The details of the operation of the components and configurable parameters are presented below.

B. ns3-Continuum Components

1) Workload Model: Client's workload is modeled as a DAG of tasks, denoted `DagTask`, whose design is inspired by Apache Airflow's [14] DAG workflows. The `DagTask` is a *container* that enables modeling composite processing pipelines as graphs defining task dependencies. `DagTasks` facilitates two kinds of dependencies: control-flow dependencies and data dependencies. The former occurs when `Task` B must run after `Task` A while the latter indicate `Task` B must run after `Task` A as it requires the data produced by `Task` A. When all the tasks within a `DagTask` are completed, the `Orchestrator` returns the results for any sink tasks, which are tasks with no successors. Supporting multiple sink tasks within a single `DagTask` facilitates a flexible framework for partial processing pipeline splitting. In such cases, results are returned by the `Orchestrator` for each sink `Task`. ns3-continuum provides methods to validate user generated workload (e.g., cycle free) and the user is expected to use them before using the `DagTask` in a simulation.

A ***task*** is the unit of work that runs on an `Accelerator`. The `Task` class defines a common interface for all task types. Every task stores metadata including randomly generated input and output sizes in bytes and compute demand in FLOPS. Additionally, metadata includes an optional deadline, a priority, and a required accelerator type string (Default: none). A task's metadata is used to record compute/backend timings populated after execution for post processing. Task lifecycle is tracked by a set of pre-defined states (e.g., created, submitted, running, ..., completed) that are stored in a ns3 `TracedValue`. This grants observability into every transition without polling. This metadata can be used by the `Orchestrator` to implement advanced scheduling policies, by the `Accelerator` during task processing, and for simulation post-processing.

2) Client: `OffloadClient` represents the key traffic generator and is designed to support a wide variety of applications. Specifically, configurable in several ways to support different simulation objectives. `DagTask`'s inter-arrival time is modeled as a random variable allowing any arrival pattern from periodic (default at 30 HZ) to bursty workloads. Additionally, adaptive client behavior is enabled through a plugable `ClientSubmissionPolicy` that controls the transmission of generated workloads. The default policy is `DropWhilePendingSubmissionPolicy` which drops generated tasks if no response is received for the last transmitted workload. The client maintains a `ClientState` struct recording historic state (e.g., sent, dropped, rejected, violated) of generated tasks, which are useful for capturing the workload dynamics during the simulations. The task violation is determined based on a configurable delay budget. Typically, the clients pass the `ClientState` struct to a `ShouldSubmit` method that cross-checks against the configured submission policy and return a boolean. Our client also uses a highly configurable `ConnectionManager` that facilitates tuning the underlying transport protocol and number of connections. The client integrates a configurable `ConnectionManager` (defaults to TCP but also supports UDP). The client implements a `GracePeriod` that enables altering typical default ns3 application stop to keep the client connection open so an in-flight workload response isn't lost at the end of the simulation.

3) Orchestrator: The `Orchestrator` is the central coordination point of resource management through pluggable `AdmissionPolicy`, `ClusterScheduler`, and an optional `DeviceManager`. The `Orchestrator` maintains a registry of backend nodes, addresses, and accelerator types and a `ClusterState` capturing per-backend load and device telemetry. Admission follows a two-phase process. First, `HandleAdmissionRequest` rejects empty or cyclic DAGs outright, and otherwise calls `AdmissionPolicy ShouldAdmit`(dag, cluster, clusterState). The orchestrator forwards the scheduling decision to the client and stores the resource assignment record in the `ClusterState`. This record include `DispatchedTaskInfo{workloadId, dagIdx, taskType, assigned node}` keyed by the tasks ID. Four scheduling logic are shipped with the tool including `RoundRobinScheduler` that simply assigns requests to available nodes in a rotating fashion and `LeastLoadedScheduler` that assigns the task to the backend leading to the minimum computing delay based on the enqueued and arriving task. `NetworkAwareScheduler` decision is also based on the minimum combined delay of computing and communication. `TierAwareScheduler` schedules workloads to closer tiers (e.g., edge) before spilling the load to deeper tiers when the task delay exceeds a configurable overflow threshold. It reschedules workload back to the closer tier when it task delay drops below a lower threshold. The `DeviceManager` is responsible for performing necessary device reconfigurations (e.g., scaling the compute capacity) de explained below. At data plane, the orchestrator forwards tasks to the assigned backend nodes and track dependency until all tasks are completed. It then compiles the response based on `DagTask` configuration and sends it back to the client.

4) Accelerators: To support accelerator heterogeneity, we implement `Accelerator` as an interface to facilitate custom extensible implementation while preserving a common abstraction for accelerator behaviour. `Accelerator` interface defines (`SubmitTask`, `GetQueueLength`, `IsBusy`, `GetUtilization`, `GetComputeRate`, `ApplyState`, `GetNumPerformanceStates`); it owns an optional `EnergyModel` to track the energy consumption over time. `GpuAccelerator` represents a shipped, concrete implementation that defines multiple Operating-Performance-Points (OPP) including (frequency, voltage) pairs representing GPU DVFS. `GpuAccelerator` delegates task queueing to a pluggable `QueueScheduler` (`FifoQueueScheduler` by default) and per-task timing to a pluggable `ProcessingModel`. `SubmitTask` enqueues the task and if idle, calls `StartNextTask`, which dequeues the next task and invokes `ProcessingModel::Process(task, this)`. The processing time is typically estimated based on the task compute load and the processing capacity scaled by

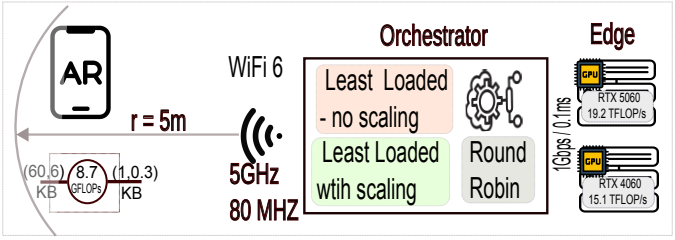


Fig. 2: Edge Offloading Setup

a compute efficiency. This design enables tracking queuing and processing delay for every task. The `DeviceManager` evaluates scaling decision on every task dispatch and completion. If the returned OPP differs from the backend's current one, the device manager triggers the change on the backend through the accelerator `ApplyState(idx)`. We ship two ScalingPolicy including `UtilizationScalingPolicy` that alternates between highest and lowest frequency to serve the workloads in the fastest manner when received and `ConservativeScalingPolicy` that gradually increment or decrement the speed to meet a configurable target drain rate. Note that a frequency change mid-task only impacts the unprocessed part of the task, so a DVFS command can be applied safely to an in-flight task.

IV. Performance Evaluation

In this section, we present two evaluation scenarios showing the capabilities of ns3-continum. The first presents a typical multi-user edge-assisted AR over WiFi 6 with simple tasks and illustrates DVFS for GPU accelerators. The second illustrates composite workload scheduling in a two-tier computing continuum.

A. Edge offloading scenario

We consider n Augmented Reality (AR) users that offload their intensive computation to an edge server over WiFi 6. The access point operates at 5GHz with 80 MHz bandwidth and employs a single user scheduler. The users were randomly positioned on a disk around the access point with radius 5 `m`. The clients generate the frames periodically at a rate of 30 fps. DagTask is composed of a single task for each AR frame. The `computeDemand`, `inputSize`, and `outputSize` of this task are sampled from bounded `NormalRandomVariable` distributions, as detailed in Table II. The input data size represents a compressed video frame transmitted for inference, with a 640×640 JPEG image, while the output size represents the bounding boxes' information of the detected objects. The clients consider `DropWhilePendingSubmissionPolicy` to avoid explosive response latencies in the presence of compute or communication bottlenecks. Hence, the client drops generated frames at the source.

We consider two worker nodes with different Graphics Processing Units (GPUs)s: the Nvidia RTX 5060 and Nvidia RTX 4060, whose specs are summarized in Table III. We assume the provided 5 Operating Performance Points (OPPs) for DVFS validation as NVIDIA does not publish discrete operating point tables for consumer GPUs.

TABLE II: Standard deviations used for workload parameters

Parameter	Mean	Standard deviation
Input size	60000	6000
Compute demand	8.7×10^9	8.7×10^8
Output size	1000	300

TABLE III: GPU Parameters

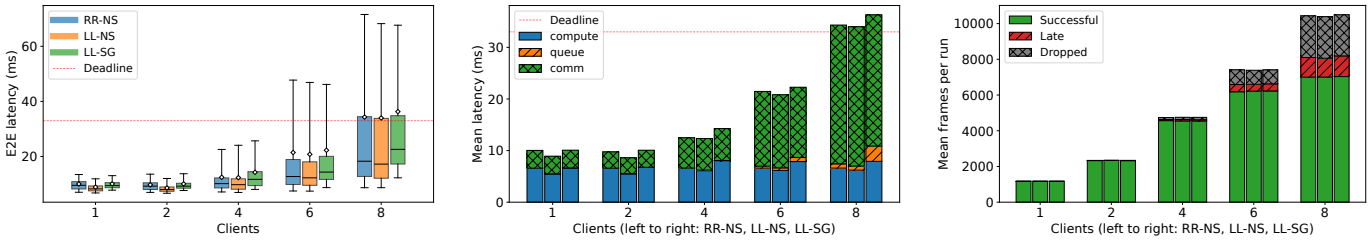
Parameter	RTX 5060	RTX 4060
Peak FP32 Rate (TFLOP/s)	19.2	15.1
Transfer Bandwidth (GB/s)	32	16
Compute Efficiency η	7.5%	7.5%
Effective Capacitance C_{eff} (nF)	18.3	16.6
Static Power P_{s} (W) [15]	9	13
Frequency Range (GHz)	2.28–2.76	1.83–2.46
Voltage Range (V)	0.88–1.04	0.88–1.03

In this setup, we vary the number of clients in the network and compare the performance of three orchestrator scheduling strategies
- Round-robin with no GPU scaling (RR-NS)
- Least-load with no GPU scaling (LL-NS)
- Least load with GPU scaling (LL-SG)

For all scheduling algorithms, scheduling decisions are taken for every offloaded workload (task). The reported results are gathered from 10 runs for each configuration.

Fig. 3a plots the end-to-end (E2E) latency distribution vs. the number of clients for different scheduling strategies. For all strategies, the latency statistics increase with the number of users. The LL-NS achieves the least average and median latency. LL-NS captures the processing heterogeneity of the workers' GPUs and balances the assigned load between the two workers when compared to HW-agnostic strategies, i.e., such as RR-NS. The figure also indicates that vertical DVFS in the LL-GS strategy increased E2E latency statistics, especially as the number of users grows. The figure also shows that the simulated system reliably serves up to four users beyond which the E2E latency of some frames exceeds the delay bound.

To identify the root cause of the delay explosion, we analyzed the delay components of the E2E latency as illustrated in Fig. 3b, which plots the average computing and communication delays. The computing delay remains stable with a minor increase in the GPU queuing delay. On the other hand, the average communication delay monotonically increases with the number of users due to increased traffic load and user contention. It is important to note that the workload scheduling strategy impacts the traffic dynamics, as the client submission policy is tied to frame latency. Fig. 3c plots the frame processing outcome versus the number of clients for the tested scheduling strategies. At low client counts, the system is over-provisioned, and the majority of AR frames overcome the inherent system network and workload randomness. With 4 clients and more, late and dropped frame percentages continuously grow as high E2E latency exceeds



(a) End-to-end latency distribution (b) Average Latency components (c) Frame outcomes

Fig. 3: Performance metrics for Edge Offloading scenario

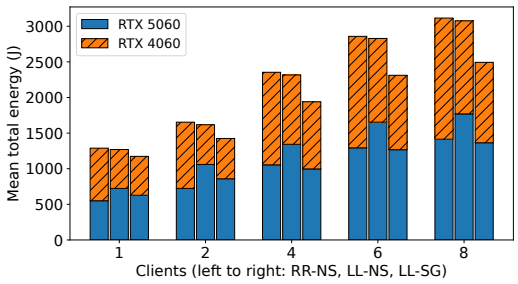


Fig. 4: Mean total energy per backend for Edge scenario

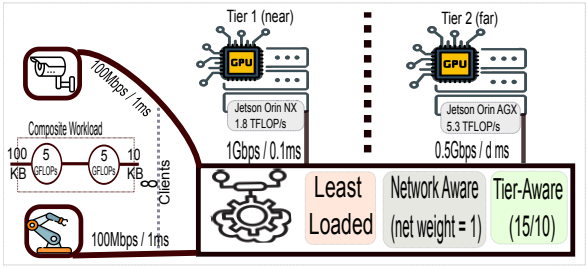


Fig. 5: Multi-tier Continuum Experiment Setup

the frame inter-arrival period, triggering frame drops. RR-NS has the largest number of dropped frames at 8 clients as it overloads the RTX 4060 while LL-NS has the lowest number of late and dropped frames.

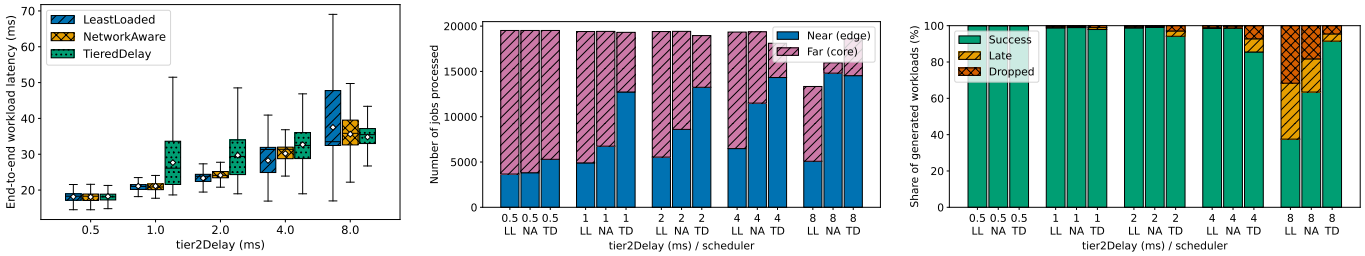
Fig. 4 compares the energy profiles of the simulated scheduling schemes as the number of clients increases. LL-SG consistently achieves the least total energy consumption. With 8 clients, LL-SG consumes $\sim$ 2500 J with both LL-NS and RR-NS approximately consuming $\sim$ 3100 J. All scheduling schemes led to uneven energy consumption for GPU servers. LL-NS allocates $\sim$ 1800 J to the RTX 5060 and $\sim$ 1300 J to the RTX 4060. In contrast, RTX 4060 consumes $\sim$ 1700 J in comparison to RTX 5060's $\sim$ 1400 J with RR-NS as the former features higher utilization for the same number of tasks. The experiment illustrates the tradeoff between energy consumption and service quality (latency) motivating the development of optimized scheduler for DCSs.

B. Multi-tier Continuum Offloading

This experiment considers a composite workload offloaded to a two-tier computing continuum with the setup parameters illustrated in Fig. 5. The workload involves two sequential tasks each featuring an average compute load of 5 GFLOPs, the average dag input data size of 100KB, and the average output data size of 10KB. All the workload parameters are uniformly distributed with their bounds set to 10% from the mean. We consider 8 clients generating the workload periodically at 25 HZ. Each client is connected to the orchestrator with a (100 Mbps, 1 ms) link. The first tier represents a local edge edge computing operating a Jetson Orin NX (1.8 TFLOP/s) and is connected to the orchestrator by a link with 1Gbps capacity and 0.1ms propagation delay. The second tier represents a core computing tier operating a Jetson Orin AGX (5.3 TFLOP/s) and is connected to the orchestrator by a link with 0.5 Gbps capacity and configurable propagation delay. We vary the delay between 0.5 ms to 8 ms. These values represent typical delays for private and regional deployment for the network core. The presented results are collected for 10 runs each simulating 5 sec.

We compare the performance of Least-loaded, network-aware, and tier-aware scheduling policies as we vary the delay between the orchestrator and tier two server. Least-loaded decisions are solely based on the task compute delay while network-aware scheduler decision is based on both computing and communication delay for each tier. Tier-aware delay schedules all tasks to edge until the task delay is estimated to exceed an overflow delay set to 15ms and reschedule tasks back to edge again when the estimated edge backend delay drops below a lower threshold, set to 10 ms in this experiment. These thresholds should be selected based on the expected computing delay component in the total delay budget.

Figure 6a plots the E2E latency boxplots versus the second-tier propagation delay for different scheduling algorithms. At the lowest latency (0.5 ms), all scheduling algorithms achieve similar latency statistics. Figure 6b also shows that both network-aware and least load have nearly similar load distribution between the core and edge, illustrating the insignificant impact of network delay knowledge. All algorithms achieve 100% successful processing of offloaded workloads 6c. As the delay increases, the average workload latency follows the trend for all scheduling algorithms. Tier-aware strategy experienced the highest delay statistics (average and percentile) for all second-tier link delay values except the 8 ms case, where the reliance on the edge led to a competitive



(a) End-to-end latency distribution (b) Workload Tier distribution (c) Workload Processing outcomes

Fig. 6: Performance metrics for Multi-tier Continuum

lower latency average. The least load strategy unsurprisingly shows a higher tendency to send tasks to the second compute-resourceful tier (6b) which worked well from latency (Fig. 6a) and success rate (Fig. 6c) for low link delay. However, workloads experience higher latency and lower success rates when the second-tier delay reaches 8 ms. Similar performance trends are observed for network-aware scheduling. However, the latter's communication delay awareness propels using the edge more to avoid communication overhead. This experiment shows that changing one system parameter has a significant impact on the performance trends and motivates leveraging ns3-continuum for scalable benchmarking of different DCS strategies before real deployment.

V. Conclusion and Future Work

ns3-continuum establishes a a modular, highly extensible framework for evaluating DCSs by bridging network simulation and compute workload execution within the robust ns-3 ecosystem. It offers a flexible, low cost testbed for rapidly prototyping and stress-testing application and network management strategies. We show that ns3-continuum serves as a versatile, scalable benchmarking instrument capable of evaluating various DCS topologies operating under realistic, composite workloads and advanced hardware accelerators. Our performance evaluations clearly demonstrate that scheduling strategies fundamentally dictate both application-level Quality of Service (QoS) and system-wide energy consumption, providing the empirical foundation needed to drive joint energy-QoS optimization models. Moving forward, ongoing development will focus on expanding ns3-continuum with state-of-the-art accelerator integrations and engineering intelligent, optimized scheduling policies to support the next generation of energy-efficient, edge-cloud continuum architectures.

ACKNOWLEDGMENT: This research is supported by Taighde Éireann – Research Ireland under Grant number 13/RC/2077 P2.

References

[1] A. Akundi, D. Euresti, S. Luna, W. Ankobiah, A. Lopes, and I. Edinbarough, "State of Industry 5.0—analysis and identification of current research trends," *Applied System Innovation*, vol. 5, no. 1, p. 27, 2022. [Online]. Available: https://doi.org/10.3390/asi5010027

[2] S. Dustdar, V. Casamayor Pujol, and P. K. Donta, "On distributed computing continuum systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 4092–4105, 2023.

[3] M. Tortonesi, "The compute continuum: Trends and challenges," *Computer*, vol. 58, no. 3, pp. 105–108, 2025.

[4] X. Su, J. Qi, J. Wang, R. Wang, and Y. Yao, "EasiEI: A simulator to flexibly modeling complex edge computing environments," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1558–1571, 2024.

[5] M. Jansen, A. Al-Dulaimy, A. V. Papadopoulos, A. Trivedi, and A. Iosup, "The SPEC-RG reference architecture for the compute continuum," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 469–484.

[6] R. M. Albana'a and N. T. Alsohybe, "Optimizing latency for augmented reality applications via mobile edge computing," in *2024 1st International Conference on Emerging Technologies for Dependable Internet of Things (ICETI)*, 2024, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/document/10777191

[7] C. Silvano, D. Ielmini, F. Ferrandi, L. Fiorin, S. Curzel, L. Benini, F. Conti, A. Garofalo, C. Zambelli, E. Calore, S. Schifano, M. Palesi, G. Ascia, D. Patti, N. Petra, D. De Caro, L. Lavagno, T. Urso, V. Cardellini, G. C. Cardarilli, R. Birke, and S. Perri, "A survey on deep learning hardware accelerators for heterogeneous HPC platforms," *ACM Computing Surveys*, vol. 57, no. 11, pp. 1–39, 2025.

[8] H. F. Sheikh, I. Ahmad, Z. Wang, and S. Ranka, "An overview and classification of thermal-aware scheduling techniques for multi-core processing systems," *Sustainable Computing: Informatics and Systems*, vol. 2, no. 3, pp. 151–169, Sep. 2012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210537911000485

[9] H. Casanova, A. Legrand, and M. Quinson, "SimGrid: A generic framework for large-scale distributed experiments," in *Tenth International Conference on Computer Modeling and Simulation*. IEEE Computer Society, 2008, pp. 126–131. [Online]. Available: https://ieeexplore.ieee.org/document/4488918

[10] R. N. Calheiros, R. Ranjan, C. A. F. De Rose, and R. Buyya, "CloudSim: A novel framework for modeling and simulation of cloud computing infrastructures and services," 2009. [Online]. Available: https://arxiv.org/abs/0903.2525

[11] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in internet of things, edge and fog computing environments," 2016. [Online]. Available: https://arxiv.org/abs/1606.02007

[12] C. Sonmez, A. Ozgovde, and C. Ersoy, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," *Transactions on Emerging Telecommunications Technologies*, vol. 29, no. 11, p. e3493, 2018. [Online]. Available: https://dl.acm.org/doi/10.1002/ett.3493

[13] Q. Lu, G. Li, and H. Ye, "EdgeSim++: A realistic, versatile, and easily customizable edge computing simulator," *IEEE Internet of Things Journal*, vol. 11, no. 21, pp. 35 341–35 360, 2024.

[14] Apache Airflow. Apache airflow. [Online]. Available: https://airflow.apache.org/

[15] TechPowerUp. Gpu static powers. [Online]. Available: https://www.techpowerup.com/review/zotac-geforce-rtx-5060-solo-8-gb/39.html